TinyFL: On-Device Training, Communication And Aggregation On A Microcontroller For Federated Learning

Lars Wulfert*, Christian Wiede* and Anton Grabmaier†
*Embedded AI Department, Fraunhofer IMS, Duisburg, Germany
† Department of Electronic Components and Circuits, University of Duisburg-Essen, Duisburg, Germany
Email: *lars.wulfert@ims.fraunhofer.de, *christian.wiede@ims.fraunhofer.de, †anton.grabmaier@uni-due.de

2023 21st IEEE Interregional NEWCAS Conference (NEWCAS) | 979-8-3503-0024-6/23/$31.00 ©2023 IEEE | DOI: 10.1109/NEWCAS57931.2023.10198040

***Abstract*—In federated learning (FL), in contrast to centralized ML learning processes, ML models are sent rather than the raw data. Therefore, FL is a decentralized and privacy-compliant process currently experiencing significant research interest. As a result, initial investigations were carried out with FL and microcontrollers (MCUs). However, each of these studies used a PC as a server. In this work, we introduce TinyFL, a method using only MCUs to build a low-cost, low-power, and low-storage system. TinyFL uses a hybrid master/slave protocol where the master MCU is responsible for communication and aggregation. Thereby, the communication is performed by inter-integrated circuit (I²C). TinyFL demonstrates that communication and aggregation for FL can be performed on only MCUs. Furthermore, we show that the training with TinyFL is 11.57 % faster than centralized training using a gesture recognition use case.**

***Index Terms*—Federated Learning, Microcontroller, On-Device Aggregation, On-Device Training, Embedded Systems, TinyFL**

I. Introduction

In machine learning (ML) applications, the accuracy of the ML model relies on the quality and amount of data available for training. The data is often collected using internet of things (IoT) devices, which will continue to grow in the coming years. One trillion IoT devices are expected to be in use by 2035 [1]. However, the high communication with a cloud over the internet leads to several problems. Firstly, a massive amount of raw data has to be transferred to train the ML model or predict the transferred data. Feature extraction can be used on the data before it is sent to reduce the transferred data, whereby information can be lost. Secondly, the devices require a stable internet connection to send data to a cloud, which could not be valid at all times. Lastly, for a wide area of applications, it is not allowed to share raw data due to legal regulations since unauthorized individuals could access personal and confidential data. McMahan et al. [2] introduced FL to overcome these limitations. Unlike traditional ML methods, FL allows for training ML models without transferring raw data to the cloud instead, the ML models are shared. Although the devices share no raw data, it was already demonstrated that it is feasible to reconstruct the raw data from the ML models [3], [4]. Cryptographic methods [5]–[10] or data perturbation techniques [11], [12] are used to increase the privacy of shared data. However, using these methods involves a tradeoff between accuracy of the ML model, communication costs, and privacy [13]. Therefore, FL was investigated on MCUs without internet connection [14]–[16]. However, these works used a PC as a server, which is not a low-power, low-cost and low-storage setup. The PC controlled the communication and performed the aggregation. Although a PC was used, it was noted in [14] that the communication took almost one minute.

To overcome the communication limitations, we propose TinyFL. TinyFL uses only MCUs to execute the entire FL process. Therefore, TinyFL does not share its ML models with a PC, cloud, or a device connected to the internet. Also, no wireless communication network is used since the data distribution is carried out via I²C. Therefore, one MCU controls hierarchical communication and performs aggregation. In addition, the device that controls the communication and performs the aggregation also trains a local model with the collected data. Thus, TinyFL introduces a different approach to enhance the shared ML model's privacy and reduce energy consumption. Finally, we demonstrate TinyFL with a gesture recognition use case using low-cost, low-power, and low-storage MCUs, proving that high-performance hardware is unnecessary for communication and aggregation in FL.

II. Background And Related Work

FL was firstly introduced in 2017 to enhance the data privacy and to reduce the data transfer [2]. In FL, the training takes place on the devices on which the data was collected. Therefore, the raw data of a device must not be sent to any central processing unit. Instead, only the local ML models of a device, e.g., the weights or gradients, are sent to a central device after training. Afterward, the local models are aggregated into a global ML model. Subsequently, the global ML model is sent to all devices participating in the training [2]. This allows the use in privacy-concerned environments such as medical or industrial. In addition to FL, McMahan et al. also introduced the aggregation algorithm FederatedAveraging (FedAvg) [2]. In FedAvg the new global model is derived using the weighted average of the local models.

Subsequently, FL was further investigated and developed in many aspects. Some have further improved the privacy [9], [11], communication cost [17], [18] or whereas some have more focused on applications for FL using edge devices

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:22:26 UTC from IEEE Xplore. Restrictions apply.

[19]–[21]. Besides mobile devices various investigations and optimizations were done for IoT devices [7], [22] which resulted in the use of MCUs. The feasibility of performing FL on MCU could be demonstrated in [14], [15], and [16], using MCU as clients. Wang et al. [16] demonstrates that FL can also be used for malicious intent since FL preservers the anonymity of participant's. They used an 8-bit Atmel ATxmega128D4 MCUs for training and victim boards and performed side-channel attacks on them to obtain a subkey of Advanced Encryption Standard (AES). It could be demonstrated that FL achieved the highest probability of recovering a subkey. Although, it is not clear what server they use as their aggregator. In [14] federated transfer learning in the section of a binary image classification was performed on two MCUs. A pre-trained convolutional neural network (CNN) was used for feature extraction, and FL was used to train the flatten layer for classification. In [15] keyword spotting was performed using FL training a fully connected neural network (FCNN) on three Arduino Nano 33 BLE Sense. In both works, a PC acted as a server that performs the aggregation and was also responsible for communication. Kopparapu et al. [14] noted that the training process should be optimized in future steps. Therefore, Diogo et al. [1] developed an optimized stochastic gradient descent (SGD) especially for FL and MCU. They introduce a lightweight implementation of SGD (L-SGD) by considering that weights associated with the same output neuron share terms in the chain rule of error propagation. The training of L-SGD is 4.20 times faster and requires only 2.80 % of the memory compared to SGD.

Grau et al. [15] pointed out that the bandwidth for communication between the MCUs and the server was insufficient because the serial port was used for communication. Thus, the model takes longer from the server to the client than from the client to the server. Kopparapu et al. [14] also stated that the communication costs should be improved, it took one minute for the update and return path. Therefore, it was emphasized to optimize the communication protocol so that the reliability would be improved. However, the accuracy also deteriorated when more devices participated in the process. Hence, a different connection method than the serial interface is required, and a new communication protocol that increases the reliability. Also, a PC has been used for communication and aggregation, which inhibits the communication process, as Grau et al. [15] has shown. Therefore, we introduce TinyFL, which uses only MCUs, wherefore a novel communication protocol has been invented.

III. Method

For the communication between the MCUs a hybrid master/slave communication protocol is used where the master is set with the control over the communication and also the aggregation of the global model.

The proposed method is depicted in Figure 1 and processed sequentially. Before the FL process starts, the master initializes a global model with random weights. Afterward, the proposed method is executed in a loop of n communication rounds.

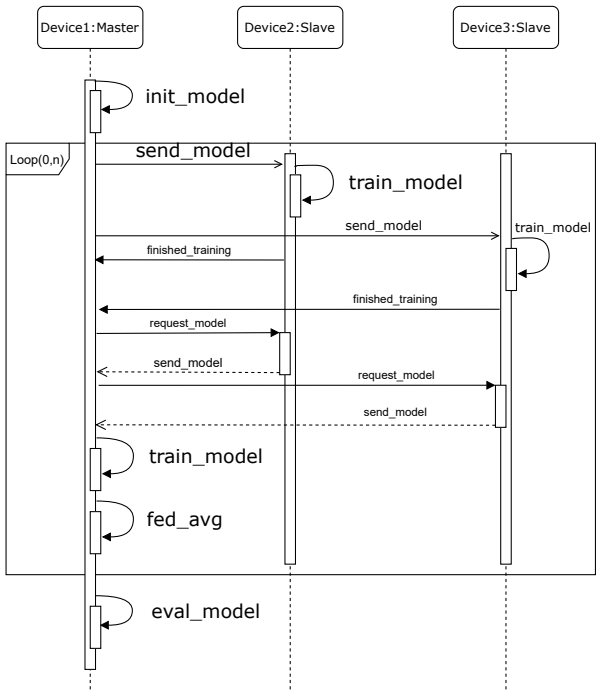


Fig. 1. Overview of the proposed method with an example of three devices.

First, the master sends the global model to the slaves. After sending the global model to the slaves, the master switches to slave mode and waits until the slave devices signal that their training is complete. The slave devices continue training the global model on their collected data if they collect enough data, which results in the local model. Whether enough data is collected can be adjusted by a threshold set before compiling. After each slave device has completed the training, it switches to den master mode and sends a signal to the master device that the training process has been completed. Once 80 % of the participating slave devices of the communication network have sent the signal that they are done with the training, the master switches back into master mode and the slave into slave mode. Following this, the master requests the local models of the slave devices and receives the local models of the slave devices. After receiving the local models, the master trains its local model with the data that it has collected. Afterward, the master aggregates the local models using FedAvg. Subsequently, the FL process starts from the beginning. After the loop has passed n rounds, the FL process is completed, and the final global model is evaluated on the master with the test dataset.

IV. Experimental Results and Discussion

In this section, we evaluate centralized training and compare it with the proposed method using non independent and identically distributed (non-IID) data and discuss the results.

A. Experimental setup

We conducted experiments using three STM32F446 MCUs with an ARM Cortex-M4. The MCUs were connected using

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:22:26 UTC from IEEE Xplore. Restrictions apply.

I²C with a clock rate of 100 kHz, using as few wires as possible. Each MCU has a flash memory size of 512 kB and an static random-access memory (SRAM) of 128 kB.

One gesture recognition dataset is used for all evaluations. The dataset comprises nine different gestures pre-recorded with the 9-axis LSM9DS1, which features a 3D accelerometer. The nine different gestures are the numerical numbers two to nine and zero, which were drawn in the air and whose movement was captured by the accelerometer. We used the data of the accelerometer's x-, y- and z-channels while recording the dataset. To provide only the most relevant information from the channels and to keep the dataset compact, feature extraction is applied to the data after it has been acquired. The feature extraction first ensures that the signals are invariant with respect to position, velocity, and direction and then performs a normalization. Subsequently, 20 data points of the signal are sampled, which are used as input parameters for the FCNN. With this method, two hundred gestures were recorded for each number. Afterward, the dataset was divided into training and test datasets, leaving 75 % of each number in the training and the remaining gestures in the test dataset.

The implementation of the MCU was done using C with the STM32CubeF4 HAL driver library [23] for I²C and Artificial Intelligence for Embedded Systems (AIfES) [24] as a library for ML. A FCNN with one hidden layer is used. The FCNN consists of 20 inputs, 12 hidden layer neurons, and ten in the output layer resulting in 382 trainable parameters. As an activation function, a ReLU is used in the hidden layer and a softmax in the output layer. The Adam optimizer with $\eta = 0.005$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 1e^{-7}$ as well as the cross entropy function is utilized for training. This results in a total memory usage of 6,608 bytes for inference and training. Also, ten epochs and a batch size of nine are used for training. At the beginning of the FL process, the global model's initial weights are not initialized randomly but statically to ensure the analyses' comparability. For this reason, the preliminary generated initial weights are stored in SRAM. Furthermore, in all investigations, the F1-Score is used to determine the accuracy of the ML models.

B. Centralized training

In order to compare the FL evaluations, we train a centralized ML model on one STM32F446. The whole training dataset was used with the same training parameters as in the FL setup. Solely the number of epochs has been changed to 50. The number of epochs is chosen, the same number carried out as communication rounds during the FL process. After training, the ML model reached an average F1-Score of all numbers of 99.79 %. The predictions of the centralized model are displayed in Figure 2 using a confusion matrix. It can be observed that the model exclusively predicted the numbers five and nine once incorrectly. The wrong prediction could be due to the same movement in the lower half of the numbers when writing the numbers five and nine in the air. The training time is 33.1 s for each epoch and 27.58 min for the complete training on the MCU.

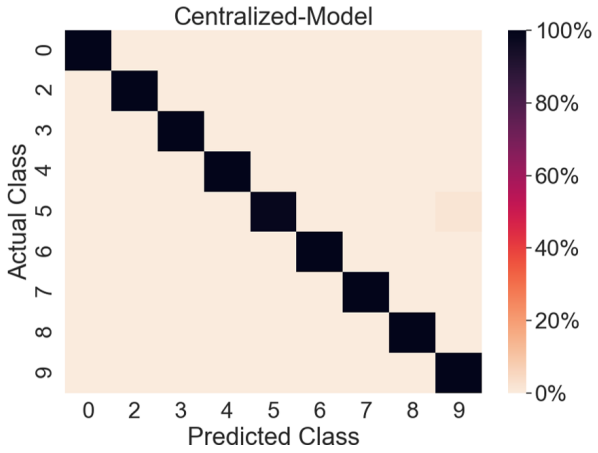


Fig. 2. The confusion matrix illustrates the prediction of the centralized model after 50 training epochs.

C. Non-IID data

In this investigation, the training's dataset is non-IID, divided into sub-datasets s_1, s_2, s_3. As a independent and identically distributed (IID) data is not a realistic data distribution and is therefore not considered. Each sub-dataset comprises three numbers. Where s_1 contains the numbers zero, two, and three, s_2 the numbers four, five, and six, and s_3 the numbers seven, eight, and nine. A different sub-dataset is stored on each device. The master receives s_1, slave 1 s_2, and s_3 is stored on slave 2. In each communication round, three gestures are selected from each number and used as input parameters for training. As a result, no data is used twice while training and overfitting can be avoided. Therefore, 50 communications rounds could be performed.

The results of the FL can be seen in Figure 3. The graph shows the average F1-Score of all numbers of the master, the slaves, and global model depicted during the communication rounds. The F1-Score of each device is similar, with the master achieving a score of 71.12 %, slave 1 at 68.08 %, and slave 2 at 73.66 % after 50 rounds. In comparison, the global model has an accuracy of 98.09 %. Hence, the F1-Score of the global model after 50 rounds is 1.70 % lower compared to the centrally trained model. Over the FL process, it can be noted that the master has a higher accuracy up to the 40th round compared to the other two devices. The master reaches a higher accuracy faster than the slaves because the numbers in s_1 differ more, so an ML model with high accuracy can be trained faster. Thus, it can also be seen that the F1-Score differs between the master and slave 2 by almost 20 % in some communication rounds up to the 40th round. The same applies to the accuracy of slave 2, where the F1-Score is mostly 5 % lower compared to the master and up to 10 % higher compared to slave 1. In addition, the accuracies of master and slave 2 start to converge as early as the 35th round. Slave 1 only converges from the 40th round onwards. Since the calculation of the F1-Score was performed with the test dataset, each model had to predict all classes of the dataset. However, the local models

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:22:26 UTC from IEEE Xplore. Restrictions apply.

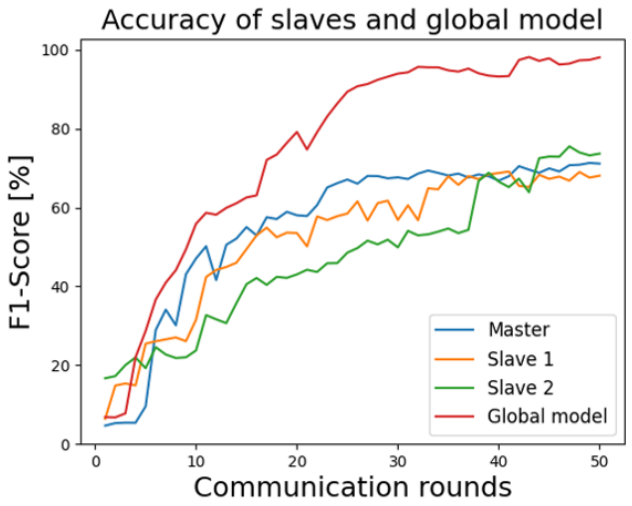


Fig. 3. The progression of the F1-Score during communication rounds of the devices and the global model is shown.

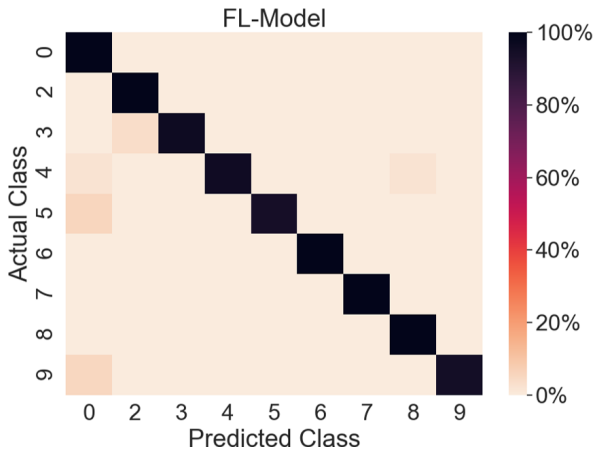


Fig. 4. The confusion matrix shows the prediction of the global model after 50 FL communication rounds.

are trained with non-IID data. Therefore the local models are trained exclusively on three numbers at a time, which temporarily degrades the accuracy of the other numbers. After aggregating the local models, the F1-Score increases, and the recognition of all numbers becomes feasible. Subsequently, the predictions of the global model after 50 communications rounds are shown in Figure 4. The figure illustrates in detail the differences compared to the centrally trained model. Compared to the confusion matrix of the centralized trained model, it can be seen that the FL models predictions are slightly worse. Thus, the number zero is incorrectly predicted (false positive) in 2.22 % of the predictions, although it is the numbers nine, five, or four. Furthermore, the number three was recognized as the number two in 0.44 % of the projections. Moreover, the number eight was predicted once, while the actual number was four. The numbers are mispredicted due to the crossover of some numbers in the motion sequence. For instance, the upper half of the nine represents a zero, causing the FL model to mispredict these numbers.

Subsequently, we investigated the proposed method's computation time, shown in Table I. The analysis has determined that sending a local ML model took 4.58 s. On the other hand, the training of a local model needed 2.12 s. The aggregation with the FedAvg algorithm required only 0.01 s. This results in a total communication round taking 29.66 s and the complete training of the whole dataset with FL taking 24.72 min. Consequently, training with FL is 11.57 % faster in comparison to the classical central training of an ML model. In addition, the privacy of the data is increased since it does not have to be sent for training. However, sending an ML model takes a long time (4.58 s). Since two local and two global models are sent, the sending and receiving of the models alone take 61.77 % of the computing time. The sending time can be reduced using a different communication protocol e.g., Serial Peripheral Interface (SPI), gossip learning, or a different aggregation algorithm e.g. FedProx [25].

However, the presented method is only designed for the pure execution of FL. Therefore, no data collection is implemented, which is necessary for actual applications. Nevertheless, there are time intervals within the training rounds where data collection can be performed. Furthermore, we analyzed the process for a total of three devices. If significantly more devices were used, the bandwidth and computation of the master would increase, which could result in a master no longer being able to act as the coordinator on its own. To this end, a multi-master system could be introduced. Also, there is currently no condition for inference to be performed on each device.

V. Conclusion

Previous attempts to use FL on MCUs still required the use of PCs as servers for communication and aggregation. To address this, we developed TinyFL, which exclusively uses MCUs for communication and aggregation, resulting in a low-power and cost-effective system. We created a hybrid master/slave communication protocol for I²C to achieve this. Our work shows that communication and aggregation can also be performed on a MCU in a reasonable amount of time. We have also shown that FL with non-IID is feasible on only MCUs, delivers similar accuracy to central training, and is even 11.57 % faster. However, we acknowledge that the scalability of this method needs further investigation. For instance, a hierarchical multi-master system using asynchronous FL may be necessary to accommodate a high number of slaves, as the communication overhead for the master may become too high.

TABLE I
Computing time of the proposed method

Function	Computation time	
	[s]	[min]
Send model	4.58	0.08
Train model	2.12	0.04
Aggregation	0.01	0.00017
One FL round	29.66	0.49
50 rounds	1483	24.72

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:22:26 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Diogo Costa, Miguel Costa, and Sandro Pinto, "Train me if you can: Decentralized learning on the deep edge," *Applied Sciences*, vol. 12, no. 9, 2022.

[2] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, AISTATS 2017*, 2017, p. 1273–1282.

[3] Ligeng Zhu, Zhijian Liu, and Song Han, "Deep leakage from gradients," in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds. 2019, vol. 32, Curran Associates, Inc.

[4] Jonas Geiping, Hartmut Bauermeister, Hannah Dröge, and Michael Moeller, "Inverting gradients - how easy is it to break privacy in federated learning?," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, Eds. 2020, vol. 33, pp. 16937–16947, Curran Associates, Inc.

[5] Yunlong Lu, Xiaohong Huang, Yueyue Dai, Sabita Maharjan, and Yan Zhang, "Blockchain and federated learning for privacy-preserved data sharing in industrial iot," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 6, pp. 4177–4186, 2020.

[6] Yunlong Lu, Xiaohong Huang, Ke Zhang, Sabita Maharjan, and Yan Zhang, "Blockchain empowered asynchronous federated learning for secure data sharing in internet of vehicles," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 4, pp. 4298–4311, 2020.

[7] Xu Cheng, Weiwei Tian, Fan Shi, Meng Zhao, Shengyong Chen, and Hao Wang, "A blockchain-empowered cluster-based federated learning model for blade icing estimation on iot-enabled wind turbine," *IEEE Transactions on Industrial Informatics*, vol. 18, no. 12, pp. 9184–9195, 2022.

[8] Fan Yang, Yanan Qiao, Mohammad Zoynul Abedin, and Cheng Huang, "Privacy-preserved credit data sharing integrating blockchain and federated learning for industrial 4.0," *IEEE Transactions on Industrial Informatics*, vol. 18, no. 12, pp. 8755–8764, 2022.

[9] Meng Hao, Hongwei Li, Guowen Xu, Hanxiao Chen, and Tianwei Zhang, "Efficient, private and robust federated learning," in *Annual Computer Security Applications Conference*, New York, NY, USA, 2021, ACSAC '21, p. 45–60, Association for Computing Machinery.

[10] Mingjun Dai, Ziying Zheng, Zhaoyan Hong, Shengli Zhang, and Hui Wang, "Edge computing-aided coded vertical federated linear regression," *IEEE Transactions on Cognitive Communications and Networking*, vol. 8, no. 3, pp. 1543–1551, 2022.

[11] Yiming Zeng, Yixuan Lin, Yuanyuan Yang, and Ji Liu, "Differentially private federated temporal difference learning," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 11, pp. 2714–2726, 2022.

[12] Shuo Wang, Surya Nepal, Kristen Moore, Marthie Grobler, Carsten Rudolph, and Alsharif Abuadbba, "Octopus: Overcoming performance and privatization bottlenecks in distributed learning," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 12, pp. 3460–3477, 2022.

[13] Muah Kim, Onur Günlü, and Rafael F. Schaefer, "Federated learning with local differential privacy: Trade-offs between privacy, utility, and communication," in *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021, pp. 2650–2654.

[14] Kavya Kopparapu, Eric Lin, John G. Breslin, and Bharath Sudharsan, "Tinyfedtl: Federated transfer learning on ubiquitous tiny iot devices," in *2022 IEEE International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops)*, 2022, pp. 79–81.

[15] Marc Monfort Grau, Roger Pueyo Centelles, and Felix Freitag, "On-device training of machine learning models on microcontrollers with a look at federated learning," in *Proceedings of the Conference on Information Technology for Social Good*, New York, NY, USA, 2021, GoodIT '21, p. 198–203, Association for Computing Machinery.

[16] Huanyu Wang and Elena Dubrova, "Federated learning in side-channel analysis," in *Information Security and Cryptology – ICISC 2020*, Deukjo Hong, Ed., Cham, 2021, pp. 257–272, Springer International Publishing.

[17] Yuhao Zhou, Qing Ye, and Jiancheng Lv, "Communication-efficient federated learning with compensated overlap-fedavg," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 1, pp. 192–205, 2022.

[18] Abeda Sultana, Md. Mainul Haque, Li Chen, Fei Xu, and Xu Yuan, "Eiffel: Efficient and fair scheduling in adaptive federated learning," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 12, pp. 4282–4294, 2022.

[19] Alexander Chowdhury, Hasan Kassem, Nicolas Padoy, Renato Umeton, and Alexandros Karargyris, "A review of medical federated learning: Applications in oncology and cancer research," in *Brainlesion: Glioma, Multiple Sclerosis, Stroke and Traumatic Brain Injuries*, Alessandro Crimi and Spyridon Bakas, Eds., Cham, 2022, pp. 3–24, Springer International Publishing.

[20] Chaoyang Zhu, Xiao Zhu, Junyu Ren, and Tuanfa Qin, "Blockchain-enabled federated learning for uav edge computing network: Issues and solutions," *IEEE Access*, vol. 10, pp. 56591–56610, 2022.

[21] Swapnil Sadashiv Shinde, Arash Bozorgchenani, Daniele Tarchi, and Qiang Ni, "On the design of federated learning in latency and energy constrained computation offloading operations in vehicular edge computing systems," *IEEE Transactions on Vehicular Technology*, vol. 71, no. 2, pp. 2041–2057, 2022.

[22] Fotis Foukalas and Athanasios Tziouvaras, "Federated learning protocols for iot edge computing," *IEEE Internet of Things Journal*, vol. 9, no. 15, pp. 13570–13581, 2022.

[23] STMicroelectronics, "stm32f4xx HAL Driver," https://github.com/STMicroelectronics/stm32f4xx_hal_driver, 10 2022.

[24] Fraunhofer Institute for Microelectronic Circuits and Systems, "AIfES for Arduino," https://github.com/Fraunhofer-IMS/AIfES_for_Arduino, 7 2021.

[25] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith, "Federated optimization in heterogeneous networks," 2018.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 16,2026 at 09:22:26 UTC from IEEE Xplore. Restrictions apply.